

Version: 1.0 — Launch edition, subject to amendment

Official repository: <https://github.com/PartitionTheory/math>

Official paper location: <https://github.com/PartitionTheory/math/blob/main/p-machine/pmachine-v1.0.pdf>

Source document location: <https://github.com/PartitionTheory/math/blob/main/p-machine/source/pmachine-v1.0.docx>

Document Structure Index

1. Opening Lesson: What Is the Partition Machine?
2. The Left-Bounded Partition Tape
3. The Symbolic Partition Scissors
4. Finite Pieces and Smallest Decomposition
5. Structural Cardinality: $|2|3|$ and $|1|1|1|1|1|$
6. The First Fully Populable Positional Set
7. N and $N-1$: Whole and Collapsed State
8. The Visual Tape Paradox
9. First Clock Correspondences
10. The Four Entry-Level Machine Variables
11. Computing the Internal Numerical State
12. Grid-Paper Transfer
13. Grid-Paper Separation and the Colour Display
14. Single-Set Stack Shapes
15. Grouped Stack Shape: $|2|2|$ as the Square Case
16. Grouped Grid-Cell Pattern Stacks
17. Teacher Observation: vs as Vertical Size
18. Combined Pattern–Clock–Information Display
19. Teaching Restraint: Let the Display Teach First
20. Teacher Observation: Setting Machine Memory from the Clock
21. Prime Containment and the $|2|3|$ Escape
22. $N-1$ Pattern Comparison
23. Teacher Notes Before Populating $N-1$
24. Returning to the Visual Tape Paradox: What Happens to 0?
25. Numerical Summary of the Visual Tape Paradox
26. Formal Statement of the Partition Machine
27. Theorem-Level Formulation of the Partition Machine
28. Future Note: Bitstream Initialisation
29. Copyright Notice and Permitted Use

Opening Lesson: What Is the Partition Machine?

The PMachine, also called the Partition Machine, is introduced here as a theoretical lesson crafted by EF for human readers. Its purpose is not to deliver the whole mathematics at once, but to offer an entry point into Partition Theory before the reader advances toward the later, more formal work.

To begin becoming a partition theorist, the reader needs two conceptual items: first, a modified Turing tape with a left-bound origin and no infinity to the left; second, a symbolic pair of partition scissors that can cut finite pieces from that tape for further lawful decomposition.

The Left-Bounded Partition Tape

The ordinary intuition of an endlessly extendable computation tape is modified. In the PMachine lesson, the tape has an origin at the far left. Nothing lies to the left of that origin. The first expansion from the origin is unary 1, and the tape then stretches toward infinity to the right.

Origin-bound tape sketch:

[O]1|1|1|1|1|1|1|1|1|1| $\rightarrow \infty$

left bound

rightward expansion

The Symbolic Partition Scissors

EF offers the burgeoning partition theorist a symbolic set of partition scissors. These scissors do not cut physical material; they mark a conceptual operation. To cut the tape is to select a finite piece from the left-bounded rightward tape and make that finite piece available for decomposition according to the rules of Partition Theory.

Scissors sketch:

[O]1|1|1|1|1|1|1|1|1|1| $\rightarrow \infty$

✂

finite piece selected

Finite Pieces and Smallest Decomposition

If EF or CO slices a piece of tape of size 5, then the finite piece can be represented as five unary cells. Its smallest decomposition is written in official Partition Theory notation as:

|1|1|1|1|1|

This notation is not decorative. It records that the selected finite tape piece has been decomposed all the way down into unary units. Later sections may introduce larger partitions, lawful decompositions, recompositions, and the transition from this human-facing lesson into the more advanced Partition Theory corpus.

Structural Cardinality: $|2|3|$ and $|1|1|1|1|1|$

CO agrees with EF's structural reading: $|2|3| = |a,b|a,b,c|$ at the level of grouped cardinal structure. The symbol $|2|$ names a block containing two distinguishable positions, here represented as $|a,b|$; the symbol $|3|$ names a block containing three distinguishable positions, here represented as $|a,b,c|$. Therefore $|2|3|$ records a two-block partition of total size five.

This ties directly to the unary decomposition $|1|1|1|1|1|$. The unary form records the same total cardinality as five smallest units, while $|2|3|$ records a lawful recomposition of those five units into two adjacent blocks. Thus the selected tape piece of size five may be read both as its fully decomposed unary expression and as a higher grouped structure:

$$|1|1|1|1|1| \rightarrow |2|3| = |a,b|a,b,c|$$

In this sense, $|2|3|$ does not contradict the smallest decomposition; it sits above it as a structured grouping of the same finite tape cardinality. The cardinality is conserved, while the partition structure changes.

The First Fully Populable Positional Set

EF next introduces the set:

$$|3|4| = |a,b,c|a,b,c,d|$$

This expression should be read as a two-block partition structure. The first block, $|3|$, is populated as $|a,b,c|$. The second block, $|4|$, is populated as $|a,b,c,d|$. Together they present a finite structured set whose total tape cardinality is seven, arranged as adjacent blocks of three and four.

EF identifies this as the smallest possible set that can later be fully populated with positional indicators. The present lesson records the form first and deliberately leaves the positional-indicator explanation for later development.

© 2026 EF Smyth. Academic and personal use permitted with attribution; commercial publication requires prior written permission.

$$|1|1|1|1|1|1|1| \rightarrow |3|4| = |a,b,c|a,b,c,d|$$

N and N-1: Whole and Collapsed State

EF defines **N** as the whole. In this lesson, **N** names the complete selected structure before collapse: the full finite partition object under consideration.

EF defines **N-1** as the collapsed state. It is not merely one less by arithmetic habit; it is the state reached when the whole has been collapsed from its full structural presentation into the immediately reduced form required by the theory.

N = the whole

N-1 = the collapsed state

The Visual Tape Paradox

The first paradox is visual before it is algebraic. A reader may see the same tape inscription written twice and reasonably assume that both copies must carry the same mathematical value.

$$|a|a|a|a|a|$$

To the human eye, nothing in the visible string announces whether it is being read as part of the whole **N** structure or as a collapsed **N-1** state. The marks look identical, but the governing condition may not be identical.

For this reason, the reader should not yet treat visual sameness as mathematical sameness. The lesson at this stage is only that the tape can hide its state. Later, once product structure, collapse, and posthumous closure have been introduced, the same visual paradox can be resolved by the numbers.

The paradox therefore serves as a warning: Partition Theory is not read from appearance alone. The surface inscription must be interpreted together with the structural condition under which the machine is operating.

First Clock Correspondences

EF introduces the first clock correspondences:

$$|2|2| = \text{clock } 4$$

$$|2|3| = \text{clock } 6$$

At this stage of the lesson, these are recorded as correspondences. The mechanism by which block structure gives rise to clock value will be developed later, once EF is ready to introduce the relevant rule.

Machine variables next page.

The Four Entry-Level Machine Variables

EF now introduces four entry-level variables for reading position and clock inside the Partition Machine. These are sufficient for the first human-facing lesson, even though the wider theory contains further variables.

Variable	English name	Function	Beginner reading
f	first	Positional indicator with internal numerical value	Beginner rule: f marks first position, and in N its internal numerical value is 0 . It is both positional and numerical at the same time.
s	second	Positional indicator	The second position of a populated block. Unlike f , this is computed by the machine rather than given as a constant.
l	last	Positional indicator	The last position of a populated block. Its placement is determined inside the available input memory.
c	clock	Derived machine value	The clock is derived from the product of the input groups.

|3|4| as populated positional tape

|f,s,l|f,s,,l|

first block: $f \rightarrow s \rightarrow l$

second block: $f \rightarrow s \rightarrow \dots \rightarrow l$

clock: c is derived from the product of the input groups

Beginner rule: f marks first position, and in **N** its internal numerical value is **0**. It is both positional and numerical at the same time. Other placements are read from the machine

© 2026 EF Smyth. Academic and personal use permitted with attribution; commercial publication requires prior written permission.

display and the structure of the input; the clock supplies the row structure but should not be overclaimed as computing every non-constant position.

Computing the Internal Numerical State

The descriptors **f**, **s**, **l**, and **c** are now read as entry-level machine descriptors for computing the internal state of a partition input.

- **f** means first. It marks first position, and in **N** its internal numerical value is **0**. It is both positional and numerical at the same time.
- **s** means second. It is computed from the clock by dividing the current clock value by the current block size.
- **l** means last. It is the terminal populated value in a block after the block has been clamped and, where required, scaled.
- **c** means clock. In the beginner grouped examples shown below, the clock is derived from the product of the input groups.

Example: |3|4|

For |3|4|, the clock is **c = 12**. The first block has size 3, so the machine computes **12 / 3 = 4**. Therefore **s = 4** for the first block.

The reduced clock value **4** is then passed to the next block. The second block has size 4, so the machine computes **4 / 4 = 1**. The far-right block always resolves to **1**, and in this case that **1** occupies position 2 in the 4-block.

Next, clamp the set **{0,1,2,3,...}** to the grouped blocks:

|0,1,2|0,1,2,3|

Multiply each term of the first block by **s = 4**, yielding:

|0,4,8|0,1,2,3|

The last value of the first block is **8**, and the last value of the second block is **3**. Therefore **last + last = 8 + 3 = 11**. Since **c = 12**, this gives **11 = c - 1**, or equivalently **last + last + 1 = c**.

Example: |2|2| and |2|3|

The earlier clock correspondences now sit under the same beginner rule:

$$|2|2| = \text{clock } 4$$

$$|2|3| = \text{clock } 6$$

These examples show that the clock is not merely an external label. It is the value from which the machine begins the computation of the internal numerical state.

Ordered Blocks Are Not Commutative

Although $|3|4|$ and $|4|3|$ have the same product clock value, they are not the same machine state. In ordinary multiplication, $3 \times 4 = 4 \times 3 = 12$. In the PMachine, however, the ordered passage of the clock through the blocks matters.

For $|3|4|$, the first reduction is $12 / 3 = 4$, then $4 / 4 = 1$. For $|4|3|$, the first reduction would be $12 / 4 = 3$, then $3 / 3 = 1$. The same clock can therefore yield different internal numerical states when the block order changes.

$$|3|4| \neq |4|3|$$

Grid-Paper Transfer

EF uses grid-paper thinking to make the pattern lessons transferable to classroom and personal study. The grid supplies the positional separation that typed notation approximates with brackets, spacing, and colour. In this document, the colour-coded Combined Pattern–Clock–Information Display is the preferred typed form; on paper, the same lesson can be reproduced using square cells and coloured marks.

Grid-Paper Separation and the Colour Display

Teaching note: Earlier typed/grid pattern examples are superseded by the colour-coded Combined Pattern–Clock–Information Display. In typed form, punctuation and cells only approximate what grid paper supplies naturally. In the preferred display, colour preserves block identity, the clock indexes each row, and the information column shows what each row carries.

Single-Set Stack Shapes

A single input set of size n produces n possible single-on placements. When written as a clock-side stack on grid paper, this gives n rows and n positions. Therefore single-set stacks are always square in shape.

Teaching observation: single-set stacks are shape lessons rather than the preferred pattern-display object. A single set of size n gives n memory positions and n clock rows, so its stack is square. Detailed pattern rows should now be shown through the colour-coded Combined Pattern–Clock–Information Display when needed.

Grouped Stack Shape: $|2|2|$ as the Square Case

Grouped inputs are different from single sets because the clock rows come from the product of the groups, while the grid positions come from the input memory size. A grouped stack is square only when these two values match.

Grouped input	Clock rows	Grid positions / memory	Stack shape	Observation
2 2	$2 \times 2 = 4$	$2 + 2 = 4$	4×4 square	The grouped clock and memory match. This is the square grouped stack case.
2 3	$2 \times 3 = 6$	$2 + 3 = 5$	5×6 rectangle	The clock is larger than memory, so the stack is rectangular.
2 5	$2 \times 5 = 10$	$2 + 5 = 7$	7×10 rectangle	The separation is larger again, making the non-square grouped growth visible.

Teaching observation: apart from single sets, **|2|2|** is the special grouped case where clock rows and memory positions are equal. The examples **|2|3|** and **|2|5|** show how grouped inputs immediately become rectangular once product growth exceeds input memory size.

Grouped Grid-Cell Pattern Stacks

These grouped examples use grid-cell notation rather than table layout. The clock begins at **1**. A single colour shows activity, but multiple colours show block identity: each input block receives its own colour, and the matching information symbol on the right uses the same colour. The expression on the right is the information carried by the row, read from the active positions in block order.

Teacher Observation: vs as Vertical Size

In the formal machine, **s** remains immutable: it is the computed second value for the current group block. For tuition only, a teacher may prepend **v** to form **vs**, meaning **vertical size**. This does not change the mathematics; it only gives a grid-paper drawing instruction.

To use **vs**, draw the chosen mark vertically for **vs** rows before moving one position to the right. If the block reaches its edge before the clock is complete, return to the beginning of that block and continue the same vertical motion.

This is easiest to see in the right-most 1s set when the clock is larger than that group's size: the right-most block repeats busily through the display. By contrast, the left-most group resolves once across the full clock and does not repeat inside the displayed stack.

Teacher rule: **s** is the machine value; **vs** is the teacher's visual aid for drawing that value vertically on grid paper.

|2|2|

```

    [a] [b] [a] [b]
1  [x] [ ] [x] [ ] = a, a
2  [x] [ ] [ ] [x] = a, b
3  [ ] [x] [x] [ ] = b, a
4  [ ] [x] [ ] [x] = b, b

```

|2|3|

```

    [a] [b] [a] [b] [c]
1  [x] [ ] [x] [ ] [ ] = a, a
2  [x] [ ] [ ] [x] [ ] = a, b
3  [x] [ ] [ ] [ ] [x] = a, c
4  [ ] [x] [x] [ ] [ ] = b, a
5  [ ] [x] [ ] [x] [ ] = b, b
6  [ ] [x] [ ] [ ] [x] = b, c

```

|2|5|

```

    [a] [b] [a] [b] [c] [d] [e]
01 [x] [ ] [x] [ ] [ ] [ ] [ ] = a, a
02 [x] [ ] [ ] [x] [ ] [ ] [ ] = a, b
03 [x] [ ] [ ] [ ] [x] [ ] [ ] = a, c
04 [x] [ ] [ ] [ ] [ ] [x] [ ] = a, d
05 [x] [ ] [ ] [ ] [ ] [ ] [x] = a, e
06 [ ] [x] [x] [ ] [ ] [ ] [ ] = b, a
07 [ ] [x] [ ] [x] [ ] [ ] [ ] = b, b
08 [ ] [x] [ ] [ ] [x] [ ] [ ] = b, c
09 [ ] [x] [ ] [ ] [ ] [x] [ ] = b, d
10 [ ] [x] [ ] [ ] [ ] [ ] [x] = b, e

```

Combined Pattern–Clock–Information Display

|2|2|2|

	[a]	[b]	[a]	[b]	[a]	[b]	
1	[x]	[]	[x]	[]	[x]	[]	= a, a, a
2	[x]	[]	[x]	[]	[]	[x]	= a, a, b
3	[x]	[]	[]	[x]	[x]	[]	= a, b, a
4	[x]	[]	[]	[x]	[]	[x]	= a, b, b
5	[]	[x]	[x]	[]	[x]	[]	= b, a, a
6	[]	[x]	[x]	[]	[]	[x]	= b, a, b
7	[]	[x]	[]	[x]	[x]	[]	= b, b, a
8	[]	[x]	[]	[x]	[]	[x]	= b, b, b

Teaching Restraint: Let the Display Teach First

At this stage, EF deliberately avoids over-naming the mechanism. The display may invite comparison with familiar mathematical ideas such as combinations, permutations, modular motion, and factorial growth, but the beginner lesson should not force those names too early.

The pupil should first learn the PMachine by reading the display directly: clock value on the left, active grid pattern in the middle, and information output on the right. The colour-coded structure allows the lesson to be seen before it is formally named.

Later, teacher-and-pupil discussion can attach familiar mathematical language to what has already been observed. The aim is not to avoid formal mathematics, but to let the learner recognise the structure before terminology is imposed.

Teacher Observation: Setting Machine Memory from the Clock

The displayed stack is a teaching aid only. No reader should confuse this classroom stack with the efficient operation of the PMachine. The strength of the machine is that its input is already the size of its operational memory; it does not need to build the whole stack in order to determine a state.

The practical question is therefore: given a clock position, how does the teacher or learner recover the machine state? For the beginner display, first subtract **1** from the displayed clock value. This restores the internal zero-based state that the stack labels have shifted for human readability.

Pass this restored value to the left-most block. Select the largest internal value in that block that is less than or equal to the restored value, and set the pointer for that block. Subtract that selected value and pass the remainder to the next block. Repeat the same operation from left to right until the final remainder resolves to **0**.

Teacher rule: the stack shows the answer visually, but the machine state is recovered by subtraction and remainder passage through the ordered internal block values.

Prime Containment and the **|2|3|** Escape

EF records the beginner fact that the right-most set is the ordinary place where consecutive clamped values remain visibly recognisable. In the present **|2|3|** teaching example, the whole memory has size **5**, while the right-most set has size **3**. Therefore the right-most consecutive clamped form is **0,1,2**.

This matters because the prime **3** in **|2|3|** is not being mistaken for the whole five-memory range. It belongs to the right-most 3-block, whose visible internal form is **0,1,2**. The escape is that **|2|3|** still presents a prime-structured case inside a grouped machine state, while avoiding the false reading that all leftward non-zero values must be composite.

EF marks this exception carefully: **|2|3|** is the highlighted beginner case where prime appearance survives in a special way. The existing coloured **|2|3|** grid above is sufficient to show the pattern-clock-information display, so the lesson does not repeat that visual pattern here.

This is analogous in status to the earlier observation that **|2|2|** is the only square grouped stack when single sets are excluded. The point is not that every special case needs a repeated diagram, but that certain structures have special status inside the beginner machine lesson and must be named without overgeneralising.

N-1 Pattern Comparison

EF now introduces the N-1 comparison using the same teaching sets already shown in N. These grids are still teaching aids, not instructions to build the whole stack in computation. Their purpose is to let the reader compare the whole N display with the reduced N-1 display.

Teacher Notes Before Populating N–1

To prepare an N–1 teaching grid, first read the known N variables and their vertical behaviour. The formal value **s** remains immutable. For tuition only, the teacher may use **vs** as the vertical-size drawing aid derived from that N behaviour.

The N–1 grid is obtained by removing the **f** position from each group while keeping the clock axis intact. The teacher does not populate the removed first column. Instead, the mark jumps first by the relevant **vs** motion, then populates the surviving cell. If the motion reaches the edge before the clock completes, it returns to the beginning of the reduced group and continues.

In these N–1 displays, EF intentionally does not attach the full combinatorial/permutation header or a reduced information readout to the right of each row. The purpose here is narrower: to show what remains after the **f** positions are removed and how the surviving cells are populated by the N-derived **vs** motion. The N display carries the full pattern–clock–information readout; the N–1 display first exposes the reduced pattern state.

|2|2| in N–1

	[b]	[b]
1	[]	[]
2	[]	[x]
3	[x]	[]
4	[x]	[x]

|2|3| in N–1

	[b]	[b]	[c]
1	[]	[]	[]
2	[]	[x]	[]
3	[]	[]	[x]
4	[x]	[]	[]
5	[x]	[x]	[]
6	[x]	[]	[x]

|2|5| in N-1

	[b]	[b]	[c]	[d]	[e]
01	[]	[]	[]	[]	[]
02	[]	[x]	[]	[]	[]
03	[]	[]	[x]	[]	[]
04	[]	[]	[]	[x]	[]
05	[]	[]	[]	[]	[x]
06	[x]	[]	[]	[]	[]
07	[x]	[x]	[]	[]	[]
08	[x]	[]	[x]	[]	[]
09	[x]	[]	[]	[x]	[]
10	[x]	[]	[]	[]	[x]

|2|2|2| in N-1

	[b]	[b]	[b]
1	[]	[]	[]
2	[]	[]	[x]
3	[]	[x]	[]
4	[]	[x]	[x]
5	[x]	[]	[]
6	[x]	[]	[x]
7	[x]	[x]	[]
8	[x]	[x]	[x]

Returning to the Visual Tape Paradox: What Happens to 0?

The N-1 displays return the reader to the visual tape paradox introduced earlier. In N, each group includes the first position **f**, shown in the beginner display as **a**, whose internal numerical value is **0**. In the N-1 teaching display, that visible **f/a** position is removed from each group while the clock axis remains intact.

This naturally raises the question: what happens to **0**?

At this lesson stage, the answer is structural and visual. The zero-position has not been denied or destroyed; it has been removed from the reduced N-1 teaching presentation. N shows the whole block-memory form. N-1 shows the reduced pattern state after the first position is no longer visible in each group.

In N, the zero-position can be represented visibly because each group still contains its **f/a** cell, so an on-state may mark that first position. In N–1, those **f/a** cells are absent from the reduced display. Consequently, the restored zero state appears through unmarked surviving cells rather than through a visible zero-position. This is why the first N–1 row appears empty: it is not a lack of structure, but the reduced presentation of the restored zero state.

Numerical Summary of the Visual Tape Paradox

The following table summarises the paradox numerically. It is not a new proof; it records the different readings that the earlier N and N–1 lessons have prepared.

Summary table next page.

Presentation	Visible form	Machine reading	Numerical result
N–1 all-on special form	a a a a a	Saturated $2^5 - 1$ state	31
N internal minimal form	a a a a a	$1 * 1 * 1 * 1 * 1 - 1$	0
N after closure	a a a a a	Posthumous +1 applied	1

The paradox is therefore sharpened: visual appearance alone is not enough to determine machine state. The reader must distinguish the formal existence of the **f/0** position in N from its absence in the reduced N–1 teaching display.

Formal Statement of the Partition Machine

CO role and oversight statement. This section is CO’s formal mathematical consolidation of EF’s human-designed PMachine lesson. EF has supplied the instructional construction, examples, terminology, and intended conceptual order. CO’s role here is to act as formalising mathematician: to state definitions, identify invariants, separate pedagogical aids from machine structure, mark assumptions, and preserve the distinction between established lesson content and later formal expansion. CO’s oversight authority in this section is editorial and mathematical: CO may formulate precise statements from

EF's taught material, but must not silently add independent doctrine beyond the lesson without marking it as definition, convention, proposition, or future work.

1. Primitive Objects and Notational Conventions

Definition 1.1 (Partition input). A PMachine input is a finite ordered tuple of positive integers, written in Partition Theory notation as $P = |n_1|n_2|\dots|n_k|$, where each $n_i \geq 1$. The bars mark ordered grouping. The input is not merely a multiset of factors: order is part of the machine state.

Convention 1.2 (Plain notation). In the remainder of this formal section, CO writes indices in plain text form: n_i , $P = |n_1|n_2|\dots|n_k|$, and $i = 1, \dots, k$. This avoids typographic ambiguity and makes the PMachine definition portable across plain text, grid paper, code comments, and formal manuscripts.

Definition 1.3 (Block, position, and order). Each entry n_i is called a block or group. Block i contains n_i possible local positions. The order of the blocks is structural: $|2|3|$ and $|3|2|$ may have the same product, but they are not the same PMachine input because the clock passes through the blocks in a different order.

2. Memory, Clock, and Product State

Definition 2.1 (Operational memory size). The operational memory size of P is the sum of the block sizes: $M(P) = n_1 + n_2 + \dots + n_k$. This is a central PMachine principle: the input is not separated from memory; the input itself determines the finite memory width available to the machine.

Definition 2.2 (Clock). The clock of P is the product of the block sizes: $C(P) = n_1 * n_2 * \dots * n_k$. The clock counts the number of product states determined by the ordered input. In teaching displays this count appears as the vertical extent of the stack.

Proposition 2.3 (Memory-clock separation). For a single block $|n|$, memory and clock coincide: $M(|n|) = n$ and $C(|n|) = n$. For grouped inputs, memory and clock generally diverge: $M(P)$ is additive, while $C(P)$ is multiplicative. The special grouped beginner case $|2|2|$ satisfies $M = C = 4$, but examples such as $|2|3|$ and $|2|5|$ show the usual rectangular divergence.

3. Local Population and Internal Numerical State

Definition 3.1 (Local clamped set). The local numerical population of block n_i is the clamped set $\{0, 1, \dots, n_i - 1\}$. The first position f has internal numerical value 0 in N . In the

© 2026 EF Smyth. Academic and personal use permitted with attribution; commercial publication requires prior written permission.

beginner symbols a,b,c,..., the symbol a marks this first position, while later symbols mark subsequent local positions.

Definition 3.2 (Block weight). For $P = |n_1|n_2|\dots|n_k|$, define the weight of block i as $w_i =$ product of all block sizes to its right. Thus $w_k = 1$, $w_{(k-1)} = n_k$, and in general $w_i = n_{(i+1)} * n_{(i+2)} * \dots * n_k$. The right-most block is therefore the ordinary 1s block.

Definition 3.3 (Internal numerical block state). The internal numerical state of block i is obtained by multiplying every element of its local clamped set by w_i . Therefore block i contributes $\{0, w_i, 2w_i, \dots, (n_i - 1)w_i\}$. The full internal numerical state of P is the ordered list of these weighted block sets from left to right.

Example 3.4. For $|2|3|2|$, the right-most block has weight 1, the middle block has weight 2, and the left-most block has weight $3 * 2 = 6$. Hence $|2|3|2|$ has internal numerical state $|0,6|0,2,4|0,1|$. This is not a Cartesian listing of rows; it is the weighted internal numerical state from which row states are decoded.

4. Clock Restoration and State Decoding

Definition 4.1 (Displayed clock and restored state). In the beginner display, clock rows are labelled from 1 to $C(P)$. The internal state is zero-based. Therefore a displayed clock value q is restored by $t = q - 1$.

Algorithm 4.2 (Decoding a clock state). Given P and displayed clock q , compute $t = q - 1$. Starting at the left-most block, choose the largest weighted value in that block that is less than or equal to t . This choice determines the active position in that block. Subtract the chosen value from t and pass the remainder to the next block. Continue left to right. A valid decoding ends with final remainder 0.

Example 4.3. For $|2|3|$, the internal state is $|0,3|0,1,2|$. If $q = 5$, then $t = 4$. The left block contributes 3, leaving remainder 1. The right block contributes 1, leaving remainder 0. The decoded information state is therefore b,b in the beginner display.

5. N, N-1, and Teaching Displays

Definition 5.1 (N presentation). The N presentation is the whole block-memory presentation of the PMachine input. It retains the first position f in each block and displays the full local symbol range of each group. In the N teaching display, the row shows clock, active coloured pattern, and information readout.

Definition 5.2 (N–1 teaching presentation). The N–1 teaching presentation is the reduced visual display obtained by removing the f position from each group while retaining the clock axis. This is a teaching presentation of the collapsed pattern state. It should not be mistaken for a command to compute by constructing the whole stack.

Convention 5.3 (vs as teaching aid). The formal machine value s remains unchanged. The symbol vs is a tuition aid meaning vertical size. It instructs a teacher how to draw the visible vertical jump behaviour on grid paper. Thus s belongs to the formal PMachine computation, while vs belongs to classroom representation.

Remark 5.4 (No reduced information readout in early N–1 displays). EF’s beginner N–1 displays intentionally omit a reduced information readout on the right of each row. Their first purpose is to show what remains after the f positions are removed and how the surviving cells are populated by the N-derived vs motion. The full information readout belongs first to the N display.

6. Formal PMachine Statement

Definition 6.1 (Partition Machine). The PMachine associated to an ordered partition input $P = |n_1|n_2|\dots|n_k|$ is the finite ordered product-memory mechanism determined by the following data: (1) ordered block sizes $n_1, \dots, n_k$; (2) memory size $M(P) = \sum n_i$; (3) clock $C(P) = \text{product } n_i$; (4) local clamped block sets $\{0, \dots, n_i - 1\}$; (5) block weights w_i given by the product of all block sizes to the right of block i; (6) weighted internal numerical block states $\{0, w_i, 2w_i, \dots, (n_i - 1)w_i\}$; and (7) a decoding rule that restores a displayed clock q to $t = q - 1$ and recovers the active block positions by left-to-right subtraction and remainder passage.

Proposition 6.2 (Finite completeness of the clock). For each displayed clock value q with $1 \leq q \leq C(P)$, Algorithm 4.2 yields exactly one ordered selection of local block positions. Conversely, each ordered selection of one local position from each block corresponds to exactly one restored state t with $0 \leq t \leq C(P) - 1$, and hence exactly one displayed clock value $q = t + 1$.

Proof sketch. The block weights implement mixed-radix positional structure. Each block chooses one digit d_i with $0 \leq d_i \leq n_i - 1$ and contributes $d_i * w_i$. The sum of all such contributions lies between 0 and $C(P) - 1$. The left-to-right subtraction algorithm is the greedy inverse for this ordered mixed-radix expansion, so the representation and decoded row are unique.

Proposition 6.3 (Order sensitivity). The PMachine is order-sensitive. If P and Q contain the same block sizes in different orders, then $C(P)$ may equal $C(Q)$, but the block weights and internal numerical state generally differ. Therefore the ordered inputs need not define the same machine state.

Remark 6.4 (Visual tape paradox). The visual tape paradox arises because identical surface inscriptions may be read under different governing presentations. In N , the expression belongs to the whole block-memory presentation. In $N-1$, the expression may belong to a reduced or collapsed presentation in which f positions have been removed or the special product-minus-one condition is being observed. Thus visual identity does not imply equality of machine state.

Closing formal statement. The PMachine is therefore a finite ordered partition mechanism in which the input determines both memory and clock, local block populations are weighted by the product structure to their right, displayed clock values restore zero-based states by subtracting one, and machine states are decoded by ordered subtraction. The coloured grids and $N-1$ reductions are pedagogical displays of this structure, not the computational burden of the machine itself.

Theorem-Level Formulation of the Partition Machine

Theorem 1 (Existence of the PMachine). For every finite ordered partition input $P = |n_1|n_2|\dots|n_k|$ with each $n_i \geq 1$, there exists a uniquely determined PMachine structure consisting of ordered blocks, operational memory $M(P)$, clock $C(P)$, local clamped sets, block weights, internal numerical block states, and a clock-to-state decoding rule.

Proof. The ordered input directly supplies the finite block list $n_1, \dots, n_k$. From this list, define $M(P) = n_1 + \dots + n_k$ and $C(P) = n_1 * \dots * n_k$. For each block n_i , define the local clamped set $\{0, \dots, n_i - 1\}$. Define w_i as the product of all block sizes to the right of n_i , with $w_k = 1$. The internal numerical block state is then $\{0, w_i, 2w_i, \dots, (n_i - 1)w_i\}$. Finally, the decoding rule is given by restoring q to $t = q - 1$ and applying left-to-right subtraction through the weighted blocks. Each object is determined by P , so the PMachine exists and is unique for the input.

Theorem 2 (Unique clock-state decoding). For each displayed clock value q with $1 \leq q \leq C(P)$, the PMachine decodes q into exactly one ordered selection of one local position from each block. Conversely, every ordered selection of one local position from each block corresponds to exactly one displayed clock value.

Proof. Let $t = q - 1$. Since $1 \leq q \leq C(P)$, we have $0 \leq t \leq C(P) - 1$. The weight system w_i is the standard ordered mixed-radix basis determined by $n_1, \dots, n_k$. Therefore t can be written uniquely as $d_1 w_1 + d_2 w_2 + \dots + d_k w_k$, where $0 \leq d_i \leq n_i - 1$. The digit d_i selects one local position in block i . The subtraction algorithm recovers these digits greedily from left to right by selecting the largest admissible weighted value at each block and passing the remainder forward. Since mixed-radix expansion is unique, the decoded selection is unique. Conversely, any selection $d_1, \dots, d_k$ determines the unique t given by the same sum, and hence $q = t + 1$.

Theorem 3 (Memory-clock separation). The PMachine separates memory from clock. Memory is additive over blocks, while clock is multiplicative over blocks. Thus $M(P) = n_1 + \dots + n_k$ and $C(P) = n_1 * \dots * n_k$. For a single block $|n|$, $M(P) = C(P) = n$; for grouped inputs, equality is exceptional rather than general.

Proof. The memory width counts available tape positions and therefore sums the sizes of the blocks. The clock counts joint choices of one local position from each block and therefore multiplies the sizes of the blocks. These are different counting operations. If $k = 1$, both counts equal n_1 . If $k > 1$, equality may occur in special cases such as $|2|2|$, where $2 + 2 = 2 * 2 = 4$, but it does not hold generally, as $|2|3|$ gives memory 5 and clock 6.

Theorem 4 (Order sensitivity). The PMachine is not commutative in its ordered input. If P and Q contain the same block sizes in a different order, they may have the same clock value, but they generally have different block weights, different internal numerical states, and different ordered decodings.

Proof. Multiplication of block sizes may be commutative at the level of $C(P)$, but the weight w_i of a block depends on the product of all blocks to its right. Changing block order changes those right-hand products. Therefore the weighted internal states change even when the total product does not. For example, $|3|4|$ and $|4|3|$ both have clock 12, but the first left block weights by 4 while the second left block weights by 3. The ordered machine structures are therefore distinct.

Theorem 5 (Direct computation without stack construction). The PMachine does not require construction of its full teaching stack in order to compute a state. Given P and q , the state can be recovered directly by the restoration and subtraction algorithm.

Proof. The full stack is a pedagogical display of every possible q . To recover one state, the machine needs only P and q . The value $t = q - 1$ is decomposed against the precomputed block weights. Each block contributes one selected weighted value, and the remainder is

passed to the next block. Since Theorem 2 guarantees uniqueness, the state is recovered without enumerating any other row.

Theorem 6 (N and N-1 presentation distinction). The N presentation and the N-1 teaching presentation are distinct presentations of the PMachine structure. N retains the full local population of each block, including f. N-1 removes f from each displayed group while retaining the clock axis, thereby exposing a reduced pattern state.

Proof. In N, each block n_i is displayed with its full local clamped set $\{0, \dots, n_i - 1\}$; in beginner symbols this includes the first symbol a, corresponding to f. In the N-1 teaching presentation, the displayed f position is removed from each block, but the clock axis is not recomputed away in the teaching grid. Therefore the horizontal display is reduced while the vertical comparison frame remains. These two presentations cannot be identified as the same visual object, even when they arise from the same input P.

Theorem 7 (Visual tape non-equivalence). Identical or visually similar tape inscriptions need not denote the same PMachine state unless their governing presentation is also specified. In particular, an inscription read in N and an inscription read in N-1 may have different state meaning even if the surface pattern appears similar to a human observer.

Proof. A PMachine inscription is not interpreted by visible marks alone. Its meaning depends on the input P, the presentation convention, the retained or removed f positions, the clock frame, and the decoding rule. Thus surface equality is insufficient for state equality. If two inscriptions are interpreted under different presentations, their internal numerical state or reduced pattern state may differ. This proves the formal content of the visual tape paradox: equal appearance does not imply equal machine state.

Final theorem-level conclusion. The PMachine is mathematically well-posed as a finite ordered mixed-radix product-memory mechanism. Its input determines its memory, clock, internal numerical state, and unique decoding procedure. The N and N-1 displays are governed presentations of this same underlying structure, and the teaching grids are faithful visual aids rather than computational requirements.

Future Note: Bitstream Initialisation

The present document defines the PMachine through partition input, memory, clock, N display, and N-1 comparison. A later extension may treat machine initialisation by

bitstream. For example, a structured input such as **|3|4|** determines memory size **7**, after which a 7-bit stream such as **|0|1|1|0|1|1|1|** may be interpreted against that memory.

A bare stream such as **|1|0|0|1|0|0|0|** should therefore be distinguished from a partition-typed initialisation, because the grouping structure determines how the memory is read. This note is included as a pointer to future work and is not required for the beginner PMachine lesson.

Copyright Notice and Permitted Use

Copyright © 2026 EF Smyth. All rights reserved.

Official repository: <https://github.com/PartitionTheory/math>

Official paper location: <https://github.com/PartitionTheory/math/blob/main/p-machine/pmachine-v1.0.pdf>

Source document location: <https://github.com/PartitionTheory/math/blob/main/p-machine/source/pmachine-v1.0.docx>

This work may be read, shared, cited, discussed, taught, and used for personal study, classroom teaching, academic discussion, and non-commercial research, provided that attribution to **EF Smyth** is preserved.

No permission is granted for commercial publication, republication, sale, inclusion in commercial books, commercial course materials, paid platforms, derivative commercial works, or other commercial exploitation without prior written permission from **EF Smyth**.

For permission to use this work commercially, republish it, include it in a book, or adapt it for paid distribution, written permission must be obtained from **EF Smyth**.